# Dukaan

**Project Profile Document** AMD DevMaster Hackathon, Track 1: Development of Multimodal Content Creation Tools

| | |
|---|---|
| Application | Dukaan |
| Team | himanshu748 (solo) |
| Hardware | AMD Radeon PRO, gfx1100 (RDNA 3), 48 GB VRAM, 48 CUs |
| Stack | ROCm 7.2.4, PyTorch 2.10.0+rocm7.2.4, ComfyUI 0.25.0 as a library |
| Models | LTX-2.3 22B (audio-video diffusion), Gemma 3 12B (text encoder) |
| Licence | Apache-2.0 |

## 1. Project background

A shopkeeper with a phone can photograph a product in ten seconds. Turning that photograph into something postable is the part that does not happen. It needs a background that is not a countertop, three different crops for three different places, and type that is legible at thumbnail size. That is an afternoon in a design tool, or a few hundred rupees to someone who owns one, per product, every time the stock changes.

The tools that promise to close this gap mostly generate the product too. That is the wrong trade. A seller cannot post a picture of a bangle that is not the bangle they will ship. Whatever the model does, the object has to survive it.

Dukaan takes one photograph and returns a set of finished creatives plus a short clip with sound, generated on a Radeon GPU, with the product carried through untouched and the words composited rather than drawn.

## 2. Target users and application scenarios

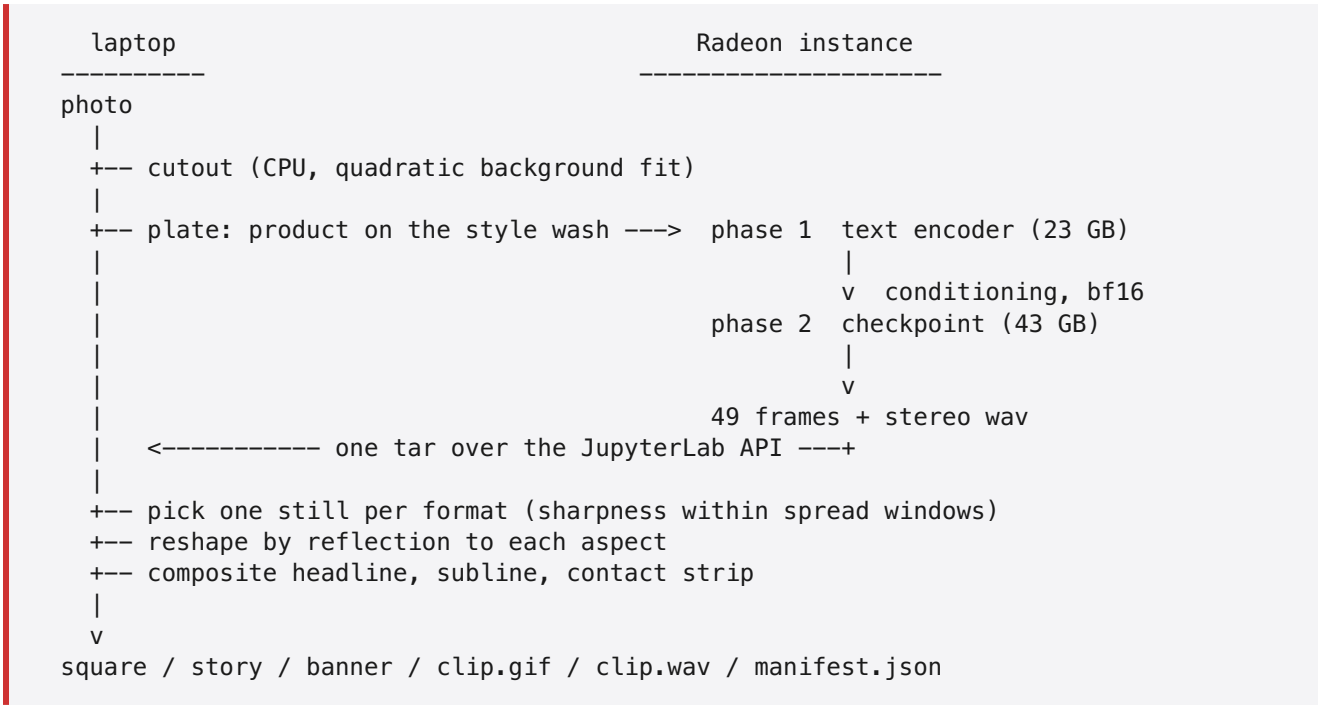
**Primary user.** A single-person or family retail business with between ten and a few hundred SKUs, selling through Instagram, WhatsApp Business, or an Indian marketplace listing. They photograph stock themselves. They have no designer and no subscription budget, and their catalogue turns over faster than they can commission artwork for it.

**Scenarios**

| Scenario | What Dukaan produces |
|---|---|
| New stock arrives, needs a feed post today | 1024x1024 square with headline, price line and contact |
| Festival push, same products, seasonal look | Same photos re-run under the `festive` style, no re-shoot |
| Story or reel slot | 1024x1820 still plus a 2-second clip with generated ambient audio |
| Marketplace or shop-page header | 1820x1024 banner |
| Price change | Re-composite type only; the generated scene is reused |

**Why the constraints matter.** Sellers photograph against whatever is behind the counter, so the background is rarely one flat colour. They type prices and phone numbers that must appear exactly as typed. They post the same product to three places with three aspect ratios. Each of these drove a specific design decision in section 3.

## 3. System architecture




**Transport.** The cloud exposes JupyterLab over HTTPS and nothing else: no ssh, no scp. Its REST API carries all three needs, so `dukaan/instance.py` uses the contents API for files, the kernels API for a process, and one websocket for that process's stdout. Long jobs are started under `setsid` so a culled kernel cannot take them down, and progress is read back from their logs.

**Split of work.** Everything cheap and deterministic stays on the CPU on purpose. The cutout is a least-squares fit; the still selection is a Laplacian variance; the reshaping is an index map; the type is a font. Spending GPU credits on any of them would be waste. The GPU does the one thing with no CPU equivalent, and it does it **once per pack** rather than once per output.

## 4. Models and algorithms

### 4.1 LTX-2.3, and why the pipeline is shaped around it

The instance carries exactly one generative model: `ltx-2.3-22b-dev`, an audio-video diffusion model, plus its Gemma 3 12B text encoder, a spatial upscaler and two LoRAs. There is no image-only model and the instance cannot reach Hugging Face to fetch one, so an image model was not an option.

That single fact set the architecture. If the only generator produces moving scenes, then stills are lifted out of a scene rather than generated one at a time. Three formats can look like three different shots because they are three different moments of the same pass, which costs one generation instead of three.

The graph mirrors the workflow that ships with the template, so the semantics match what the ComfyUI editor would run:

```
LTXAVTextEncoderLoader -> CLIPTextEncode x2 -> LTXVConditioning
CheckpointLoaderSimple -> LoraLoaderModelOnly (distilled, 0.5)
LTXVPreprocess -> LTXVImgToVideoInplace -> LTXVConcatAVLatent
  -> CFGGuider + SamplerCustomAdvanced (euler, 8-step manual sigmas)
  -> LTXVSeparateAVLatent -> VAEDecodeTiled  (video)
                          -> LTXVAudioVAEDecode (audio)
```

The sigma schedule and the 0.5 LoRA strength are taken from the shipped workflow, not tuned: the distilled LoRA is trained for that schedule.

`strength=0.7` on `LTXVImgToVideoInplace` is the one lever deliberately kept low. It is what keeps the ewer an ewer.

### 4.2 Background removal by fitting, not sampling

Backdrops are rarely one colour. A counter shot shades off as window light falls away; catalogue photography grades the sweep on purpose. Differencing against a single sampled colour keeps whichever half drifted furthest.

Dukaan fits a quadratic surface per channel to the image's border ring and differences against the fit:

- basis `[u, v, u², v², uv, 1]` on normalised coordinates, solved by least squares
- refit once with the worst quarter of residuals dropped, so a product running off the frame edge cannot drag the surface towards itself
- threshold, feather, use as alpha

A plane was tried first. It cleared the top-to-bottom wash and left an elliptical pool of backdrop exactly where the product sits, because catalogue lighting pools rather than ramps. The quadratic basis is what fixed it.

### 4.3 Still selection

Frames are split into as many contiguous windows as there are formats, and the frame with the highest Laplacian variance in each window wins. Spread first, or three formats receive three copies of one picture.

Sharpness second, because a frame caught mid-push is soft and a soft still is what a printed banner cannot hide. Frame 0 is never eligible: it is the plate the model was handed.

### 4.4 Reshaping without cropping the product

A square frame becoming a 9:16 story cannot be cover-cropped, because the product lives in the sides that would be cut. The frame is scaled to fit whole and the leftover bands are filled from its own content, **mirrored**, with the join blurred. Clamping the edge row was tried first and drew the bokeh background into long horizontal streaks.

### 4.5 Type is composited, never generated

Image models garble text, and a seller's price and phone number are the two things that must be exactly right. Headlines shrink to fit rather than truncate, because the words are the seller's. An early run also reproduced a craft channel's watermark across the bottom of a frame, so the negative prompt now names the shapes the model reaches for.

## 5. Adaptation for AMD Radeon GPU and ROCm

### 5.1 The instance cannot run the template it ships with

| | |
|---|---|
| LTX-2.3 diffusion checkpoint | 43 GB |
| Gemma 3 12B text encoder | 23 GB |
| **Total to load** | **66 GB** |
| Container cap, `/sys/fs/cgroup/memory.max` | **55 GB** |

ComfyUI's server holds every model a graph touches for the life of the process. Loading both trips the cap and the platform restarts the container mid-prompt. JupyterLab returns with zero kernels, port 8188 stops answering, and anything written outside the persistent volume is gone. It presents as a network fault.

This was diagnosed by watching `memory.current` during a load and reading `/proc/1`'s start time across a failure, which showed the container itself being replaced rather than the process dying.

### 5.2 The fix: two processes that never overlap

`scripts/radeon_ltx.py` imports ComfyUI as a library rather than starting its server, and runs the graph in two phases:

```
phase 1   load the text encoder, encode the prompts, write conditioning
          to disk, exit so the process gives its RAM back
phase 2   load the diffusion checkpoint, read the conditioning back,
          sample, write frames and audio, exit
```

Peak becomes `max(43, 23)` instead of `43 + 23`. Measured on the box:

| phase | peak container RAM | wall |
|---|---|---|
| encode | 35.4 GB | 33 s |
| sample | 51.2 GB | 55 s |
| stock template, one process | trips 55 GB, container restarts | n/a |

### 5.3 Three further adaptations

**bf16 conditioning.** The conditioning is read back while the 43 GB checkpoint is already resident, so every megabyte comes off the remaining 4 GB of headroom. Writing it in bf16 halved it, 1470 MB to 735 MB. It fell under one megabyte once the correct AV encoder replaced a plain `CLIPLoader`, which is also how the wrong encoder was caught: LTX-2.3's text embeddings carry both a video and an audio stream, and the plain loader produced a 4-D tensor the model rejected.

**VRAM eviction before the VAE decode.** ComfyUI's executor evicts models between nodes. Calling node classes directly skips that, so the 22B transformer was still resident when the VAE ran and the decode failed with 2.13 GB free out of 47.98. Calling `model_management.unload_all_models()` before the decode drops the container from 51.2 GB to 12.5 GB and frees the VRAM the decode needs. Tiled decode (512 px spatial, 32-frame temporal chunks) bounds the peak.

**`torch.inference_mode()` around the whole phase.** The LTX VAE updates the sampler's output in place, and torch refuses that across the inference-mode boundary. ComfyUI wraps every graph this way; calling nodes directly has to do the same.

`PYTORCH_ALLOC_CONF=expandable_segments:True` is set for the sampling phase to keep HIP allocator fragmentation from re-introducing the OOM.

### 5.4 Measured results

| product | style | GPU time | wall clock | output |
|---|---|---|---|---|
| brass ewer | festive | 65.3 s | 2 m 20 s | 3 stills, 49 frames, 1.96 s stereo |
| silver bangle | studio | 72.6 s | 2 m 19 s | 3 stills, 49 frames, 1.96 s stereo |
| gilt bangles | midnight | 62.0 s | batched | 3 stills, 49 frames, 1.96 s stereo |

49 frames at 768x768 with audio, per pack. Wall clock covers the plate upload, both model loads, sampling, decode and pulling everything back over the tunnel.

Transport was itself a bottleneck and a failure source: fetching 49 frames as 49 base64 requests took 3 m 34 s and the tunnel reset the connection twice mid-run. Tarring on the instance and pulling one archive brought it to 2 m 19 s. Retries now cover HTTP, the websocket connect, and the log-polling loop, so a dropped socket no longer discards a run that has already cost GPU time.

### 5.5 Two template defects worth reporting upstream

- The LTX notebook's own cell invokes `start_comfyui_with_rc_tunnel.sh`; the file on disk is `start_comfyui_with_tunnel.sh`. Run All fails.

- ComfyUI runs from `/opt/venv`, not the system python, so starting it by hand the obvious way gives `No module named 'sqlalchemy'`.

## 6. Verification

`dukaan doctor` reports what a run would actually use before it costs anything:

```
arch: gfx1100
vram_gb: 48.0
compute_units: 48
torch: 2.10.0+rocm7.2.4.git3d3aa833
card_model: 0x744b
container_ram_cap_gb: 55.0
runner: yes
ready
```

`rocm-smi` cannot reach libdrm inside this container and torch returns an empty device name, so the architecture, PCI model id and CU count stand in. They are what the driver will actually report.

26 tests run with no GPU. They cover the cases that were genuinely wrong at some point: a graded backdrop, a product touching the frame edge, a cutout with a wide transparent margin rendering tiny, `thumbnail()` refusing to enlarge, a product running into the headline band, stills landing on the same frame, and reflection versus streaking when a frame is reshaped.

Without `DUKAAN_INSTANCE` the whole pipeline runs against a CPU mock and writes real files, so the tool can be inspected before any GPU spend. The mock is never a fallback: if an instance is configured and fails, the error surfaces.

## 7. Results

Nine creatives and three clips with audio, all generated on the Radeon, are in `docs/gallery.md`. Source, tests and the runner are in the repository.